Deep Lists

A deep list in prolog is a nested structure of the form:

$$DL = [H \mid T], \text{ where } H \in \{atom, list, deep\ list\}, \text{ and } T \text{ is a deep list}.$$

The level of nesting can vary and is unknown for a deep list.

Any shallow list can also be considered a deep list.

Examples of deep lists include:

a. L1 = [1,2,3,[4]].
b. L2 = [[1],[2],[3],[4,5]]
c. L3 = [[],2,3,4,[5,[6]],[7]].
d. L4 = [[[[1]]]].
e. L5 = [1,[2],[[3]],[[[4]]],[5,[6,[7,[8,[9],10],11],12],13]].
f. L6= [alpha, 2,[beta],[gamma,[8]]].

Predicates for deep lists

1.Maximum depth – computes the maximum level of nesting in the list

```
depth([],1).
depth([H|T],R):-atomic(H),!,depth(T,R).
depth([H|T],R):- depth(H,R1), depth(T,R2), R3 is R1+1, max(R3,R2,R).
```

Queries:	
? - depth(L1,R). R = 2? ; No.	? - depth(L4,4). Yes.
? – depth(L2, R). R = 2? Yes.	? - depth(L5,R). R = 6? ; No.
? – depth(L3,R). R = 3? Yes.	? – depth(L6,4). No.

2. Flatten a deep list – returns a shallow(simple) list containing all the atomic elements form the nested list.

```
flatten([],[]).
flatten([H|T], [H|R]) :- atomic(H),!, flatten(T,R).
flatten([H|T], R) :- flatten(H,R1), flatten(T,R2), append(R1,R2,R).
```

Queries:	
? - flatten(L1,R). R = [1,2,3,4] ? ; No.	? - flatten(L,[1,2,3,4]). L=[1,2,3,4] Yes.
? – flatten(L2, R). R = [1,2,3,4,5] ? Yes.	? - depth(L5,R). R = [1..13] ? ; No.
? – flatten(L3,R). R = [2,3,4,5,6,7]? Yes.	? – flatten(L6,[alpha, 2, beta, gamma ,8]). Yes.

3. List heads – return the atomic elements from the deep list, which are at the head of a shallow list

Solution 1.

```
% skips all atomic elements, and creates a new list with the rest
skip([],[]).
skip([H|T],R):-atomic(H),!,skip(T,R).
skip([H|T],[H|R]):-skip(T,R).
% takes the first element from each list, then skips the rest of atomic elements, calls recursively for lists.
heads([],[]).
heads([H|T],[H|R]):-atomic(H),!,skip(T,T1),heads(T1,R).
heads([H|T],R):-heads(H,R1),heads(T,R2),append(R1,R2,R).
```

Solution 2.

% skips all atomic elements, when arrives at a new list, with an atomic head, takes the head, and calls recursively, for lists with non-atomic head for both head and tail(The elements immediately after a '['.

```
heads2([],[]).
heads2([H|T],R):-atomic(H),!,heads2(T,R).
heads2([[H1|T1]|T],[H1|R]):- atomic(H1),!, heads2(T1,R1), heads2(T,R2),
                              append(R1,R2,R).
heads2([H|T],R):-heads2(H,R1),heads2(T,R2),append(R1,R2,R).
%If we don't do this, we lose the first element in case it's atomic.
heads3([H|T],[H|R]):-atomic(H),!,heads2(T,R).
heads3(L,R):-heads2(L,R).
```

Solution 3.

```
%Uses a flag to determine if we are at the first element of a list
heads3([],[],_).
heads3([H|T],[H|R],1):-atomic(H),!,heads3(T,R,0).
heads3([H|T],R,0):-atomic(H),!,heads3(T,R,0).
heads3([H|T],R,_):-heads3(H,R1,1),heads3(T,R2,0), append(R1,R2,R).
heads_pretty(L,R) :- heads(L, R,1).
```

Queries:	
? - heads(L1,R). R = [1,4] ? Yes.	? - heads(L6, R]). R = [alpha, beta, gamma, 8] ? Yes.
? – heads(L2, R). R = [1,2,3,4] ? Yes.	? - heads(L5,R). R = [1..9] ? ; No.
? – heads(L3,R). R = [5,6,7]? Yes.	? – heads(L,[1,2,3,4,5]). L = [1,2,3,4,5]? Yes.

4. Member – determines the membership of an element, list or deep list to a deep list.

member1(H,[H|_]).

member1(X,[H|_]):-member1(X,H).

member1(X,[_|T]):-member1(X,T).

Queries:	
? –member1(1,L1). Yes.	? – member1(X,L4). X= [[[1]]] ? ; X = [[1]] ? ; X= [1] ; X = 1; No.
? – member1(4, L2). Yes.	? – member1(X,L6). X = alpha? ; X = 2? ; X = [beta] ? Yes.
? – member1([5,[6]],L3). Yes.	? – member1(14,L5). No.

Quizes

Q1. Define a predicate which computes the number of atomic elements in a deep list.

Q2. Define a predicate computing the sum of atomic elements from a deep list.

Q3. Define the deterministic version of the member predicate.

Problems

P1. Define a predicate returning the elements from a deep lists, which are at the end of a shallow list(immediately before a ‘]’).

P2. Write a predicate which replaces an element/list/deep list in a deep list with another expression.

P3.*********** Define a predicate ordering the elements of a deep list by depth(when 2 sublists have the same depth, order them in lexicographic order – after the order of elements).

Hint: L1< L2, if L1 and L2 are lists, or deep lists, and the depth of L1 is smaller than the depth of L2.

L1<L2, if L1 and L2 are lists, or deep lists with equal depth, all the elements up to the k-th are equal, and the k+1-th element of L1 is smaller than the k+1th element of L2

A<L, if A is a number and L is a list

A<B – the usual ordering of elements

We suppose an ordering exists between all atomic elements, e.g. they are all numbers.